

📄 INTENT.md - Hermes-Agent Setup auf Windows 11

🎯 Projekt-Ziel

Aufbau eines ****vollständig lokalen KI-Agent-Systems**** auf Windows 11 mit:

- Llama.cpp Server als LLM-Backend
- Hermes-Agent als Agent-Framework
- Optimierung für RTX 4070 Ti SUPER (16 GB VRAM)
- Tool-Calling für autonome Agent-Tasks

🖥️ System-Spezifikation

Komponente	Wert
OS	Windows 11 Pro Insider Preview (10.0.26300)
CPU	Intel Core i7-14700F (28 Threads)
RAM	32 GB
GPU	NVIDIA RTX 4070 Ti SUPER (16 GB GDDR6X)
CUDA	13.3 UMD / Driver 610.62
Speicher	1.9 TB SSD
Display	Panasonic 4K @ 60Hz

🧩 Software-Stack

Tool	Version/Quelle	Zweck
llama.cpp	b9724 CUDA 12.4	LLM-Inferenz-Server
llamastation	github.com/vico-png/llamastation	Server-Management UI
Hermes-Agent	github.com/NousResearch/hermes-agent	Agent-Framework
Modell	Gemma 4 12B Q6_K (abliterated)	LLM (provisorisch)

****Pfade:****

- Llama: `C:\llama.cpp\llama-b9724-bin-win-cuda-12.4-x64\`
- Modelle: `C:\000.mAIn\Model\01\_LLMs\GGUF\`
- Hermes: Desktop-App (Electron, `Hermes.exe`)

⚙️ Finale Llama-Server-Konfiguration

```
```bash
C:/llama.cpp/llama-b9724-bin-win-cuda-12.4-x64/llama-server.exe ^
-m C:\000.mAIn\Model\01_LLMs\GGUF\gemma-4-12B-it-abliterated-uncensored.i1-Q6_K.gguf ^
--host 127.0.0.1 --port 8080 ^
-ngl -1 ^
-c 65536 ^
-np 1 ^
-ctk q8_0 -ctv q8_0 ^
--flash-attn on ^
-b 2048 -ub 512 ^
-t 8 ^
--no-mmap ^
--jinja
```
```

Parameter-Begründung

- `-ngl -1` → Alle Layer auf GPU
- `-c 65536` → 64K Context (Hermes-Minimum!)
- `-np 1` → 1 Slot mit vollem Context (statt 4 geteilte Slots)
- `-ctk q8\_0 -ctv q8\_0` → KV-Cache quantisiert (halbiert VRAM)
- `--flash-attn on` → Beschleunigt Attention
- `--jinja` → Aktiviert Chat-Templates für Tool-Calling
- `--no-mmap` → Kompletter Load in VRAM

VRAM-Bilanz

```
Komponente	VRAM
Gemma 12B Q6_K	~9.7 GB
KV-Cache 64K (Q8_0)	~3-4 GB
Compute Buffer	~1 GB
**Gesamt**	**~14 GB** 
```

🐿 Hermes-Agent Konfiguration (`config.yaml`)

```
```yaml  
model:
 default: gemma-4-12B-it-abliterated-uncensored.i1-Q6_K.gguf
 provider: local-127.0.0.1:8080
providers:
 local-127.0.0.1:8080:
 api: http://127.0.0.1:8080/
 name: Local (127.0.0.1:8080)
toolsets:
- hermes-cli
agent:
 max_turns: 60
 tool_use_enforcement: auto
display:
 interface: cli
```
```

🐸 Gelöste Probleme

```
### Problem 1: PowerShell `curl` Syntax  
- **Symptom:** `curl` parse error mit JSON-Body  
- **Ursache:** PowerShell-`curl` = Alias für `Invoke-WebRequest`, nicht echter curl  
- **Lösung:** `curl.exe` verwenden ODER `Invoke-RestMethod` nativ  
  
### Problem 2: Doppelter llama-server Prozess  
- **Symptom:** Nur 1.6 GB VRAM frei trotz wenig Apps  
- **Ursache:** 2x llama-server.exe gleichzeitig (PID 12828 + 26516)  
- **Lösung:** `Get-Process llama-server | Stop-Process -Force`  
  
### Problem 3: Context zu klein für Hermes  
- **Symptom:** `Hermes error: Model has context window 16,384, minimum 64,000 required`  
- **Lösung:** `-c 65536 -np 1` (1 Slot mit vollem Context statt 4 geteilte)  
  
### Problem 4: Deprecation Warning  
- **Symptom:** `Setting 'enable_thinking' via --chat-template-kwarg is deprecated`  
- **Lösung:** Parameter entfernt (Gemma kann eh kein Thinking)  
  
### Problem 5: RAFT CLI Warning (Hermes-Log)  
- **Symptom:** Endlose `raft CLI not found in PATH` Warnungen  
- **Status:** Harmlos – optionales Plugin, kann ignoriert werden
```

 Funktionierende Tests

```
Test	Status
`curl http://127.0.0.1:8080/v1/models`	 Liefert Modell-Liste
Llama WebUI auf Port 8080	 Chat funktioniert
Anthropic Proxy auf Port 8081	 Läuft
Hermes Verbindung	 Identifiziert sich als "Hermes Agent"
Tool-Calling **technisch**	 Wird ausgeführt (`Searched files 6.8s`)
```

🚩 Bekannte Limitierungen

Gemma 12B + Agent-Tasks = nicht optimal

- ****Tool-Wahl:**** Wählt oft falsches Tool (z.B. "search" statt "list_directory")
- ****Output-Verarbeitung:**** Verstümmelt Tool-Ergebnisse (50 Dateien → nur 2 zerhackte Einträge)
- ****Format:**** Kaputte Pfade mit Leerzeichen (`Doctor Web \cureit.log`)
- ****Ursache:**** Gemma wurde nicht auf Agent-Workflows trainiert

🚀 Nächste Schritte (TODO)

Priorität 1: Modell-Wechsel

****Empfohlen:**** [NousResearch/Hermes-3-Llama-3.1-8B-GGUF](https://huggingface.co/NousResearch/Hermes-3-Llama-3.1-8B-GGUF)

- Speziell für Hermes-Agent trainiert
- ~6.6 GB (Q6_K) → mehr VRAM-Reserve
- Natives Tool-Calling

****Alternativen:****

- `NousResearch/Hermes-4-14B-GGUF` (Q4_K_M, ~8.5 GB)
- `bartowski/Qwen2.5-Coder-14B-Instruct-GGUF` (für Coding)

Priorität 2: Test-Suite

Nach Modellwechsel diese Tests fahren:

```

1. Liste alle Dateien in C:\Users\nikol\Desktop auf.
2. Erstelle Datei HERMES\_TEST.txt mit Inhalt "Tool-Calling funktioniert".
3. Erstelle Ordner "hermes\_test", schreibe hello.py, führe aus.
4. Wie viel RAM hat mein System? Lies aus Windows aus.

```

Priorität 3: Optimierungen (optional)

- Web-Backend konfigurieren (`web.backend`)
- TTS/STT einrichten (Edge/Local Whisper bereits konfiguriert)
- Memory-System testen (`memory.memory\_enabled: true`)
- Skills/Plugins erkunden

🗺️ Endpoints Übersicht

| Service | URL | Zweck |
|------------------|-------------------------------------|-------------------------|
| Llama OpenAI-API | `http://127.0.0.1:8080/v1` | Chat Completions |
| Llama WebUI | `http://127.0.0.1:8080` | Direkter Browser-Chat |
| Anthropic Proxy | `http://127.0.0.1:8081/v1/messages` | Claude-kompatible Tools |
| Health Check | `http://127.0.0.1:8080/health` | Status |

🏠 Wichtige Erkenntnisse

1. ****VRAM-Management ist kritisch**** – Browser, DWM und andere GPU-Apps fressen 2-3 GB
2. ****`nvidia-smi` ist der wichtigste Diagnose-Befehl für GPU-Auslastung**
3. ****`-np 1` ist Pflicht für hohen Context (sonst Context/Slots)**
4. ****KV-Cache-Quantisierung (`-ctk q8\_0`)** ist der größte Game-Changer für lange Contexts**
5. ****Modell-Wahl matters**** – nicht jedes LLM ist agent-fähig
6. ****Tool-Calling ≠ Agent-Fähigkeit**** – das Modell muss auch das richtige Tool wählen können

📅 Stand

****Datum:**** 19.06.2026

****Status:**** 🚩 Setup funktioniert technisch, Modellwechsel zu Hermes-3-8B empfohlen für produktiven Einsatz

🔗 Referenzen

- llama.cpp: <https://github.com/ggml-org/llama.cpp>
- Hermes-Agent: <https://github.com/NousResearch/hermes-agent>
- llamastation: <https://github.com/vico-png/llamastation>
- Hermes-3 Modelle: <https://huggingface.co/NousResearch>
- KV-Cache Doku: <https://github.com/ggml-org/llama.cpp/pull/16391>